

■ B.TECH · 1ST YEAR · DATA STRUCTURES

Stacks

Unit III – Complete Study Material

SUBJECT

Data Structures

UNIT

Unit III

LEVEL

B.Tech Year 1

Introduction to Stacks

LIFO Principle

Push & Pop

Array Implementation

Linked List Implementation

Expression Evaluation

Infix ↔ Postfix

Backtracking

Call Stack

Stack Overflow

Covers Complete Syllabus · Exam-Oriented · 14 Sections · Full Diagrams

Table of Contents

1. Introduction to Stacks	3
2. Properties of Stacks (LIFO Principle)	4
3. Stack Operations – Push, Pop, Peek	5
4. Stack Implementation Using Arrays	6
5. Stack Implementation Using Linked Lists	8
6. Stack Overflow & Underflow	10
7. Applications of Stacks	11
8. Expression Evaluation Using Stack	12
9. Infix, Prefix & Postfix Conversions	14
10. Backtracking Using Stack	16
11. Reversing Data Using Stack	17
12. Call Stack Concept	18
13. Questions & Answers	19
14. Quick Revision Sheet	22
• Textbook & References	23

Stack vs Queue – At a Glance

Feature	Stack	Queue
Principle	LIFO – Last In, First Out	FIFO – First In, First Out
Insert end	Top	Rear
Delete end	Top	Front
Primary operations	Push, Pop, Peek	Enqueue, Dequeue
Real-life analogy	Stack of plates	Queue at a ticket counter
Applications	Expression eval, recursion, undo	CPU scheduling, BFS
Overflow condition	top == MAX-1	rear == MAX-1
Underflow condition	top == -1	front == rear

Section 1

Introduction to Stacks

DEFINITION

A **Stack** is a linear, ordered collection of elements in which all insertions and deletions occur at one end called the **Top**. It follows the **LIFO (Last In, First Out)** principle — the most recently added element is always the first to be removed.

A stack is one of the most fundamental and widely used data structures in computer science. It can be visualised as a pile of plates on a table — you can only add a new plate on top or remove the topmost plate. You cannot access or remove a plate from the middle or bottom without first removing all plates above it.

1.1 Real-World Analogies

Stack of Plates

- Plates are placed one on top of another
- The last plate placed is the first one used
- Cannot remove a plate from the middle

Stack of Books

- Books are stacked vertically
- Reading from top downward
- Adding a book places it on top

Browser Back Button

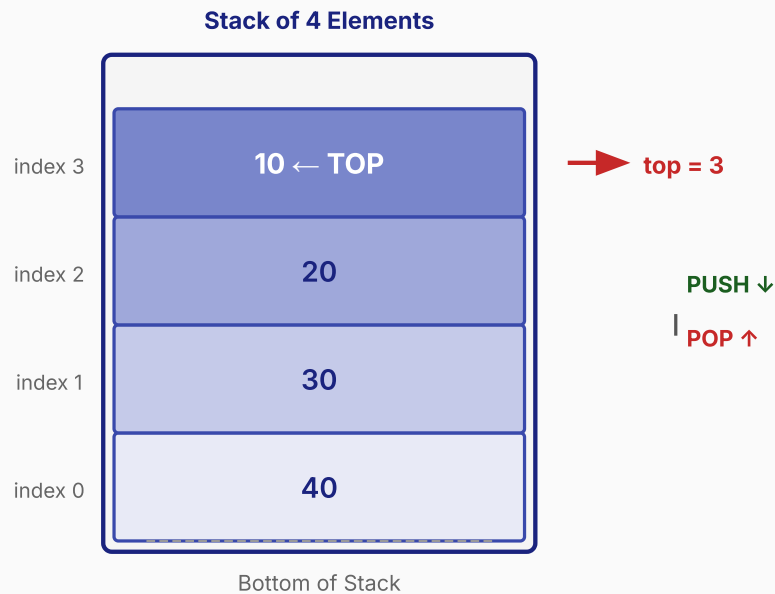
- Each visited URL is pushed onto a stack
- Pressing "Back" pops the last visited page
- History maintained in LIFO order

Ctrl+Z (Undo)

- Every action is pushed onto a stack
- Undo pops and reverses the last action
- Multiple undos go back in LIFO order

1.2 Stack Representation (LIFO Diagram)

FIG 1.1 – STACK REPRESENTATION (LIFO PRINCIPLE)



Elements 40, 30, 20, 10 pushed in order. Element 10 is at the top (last inserted, first to be removed).

1.3 Characteristics of a Stack

- **Linear structure:** Elements are arranged in a sequential order.
- **Single-end access:** All operations (insert/delete) happen only at the top.
- **Dynamic or static:** Can be implemented with fixed-size arrays or dynamic linked lists.
- **Ordered:** The order of insertion is strictly preserved for removal.

⚡ TIME COMPLEXITIES – ALL STACK OPERATIONS

Push (insert at top)	$O(1)$
Pop (remove from top)	$O(1)$
Peek / Top (view top element)	$O(1)$
IsEmpty / IsFull	$O(1)$
Search / Traverse	$O(n)$
Space Complexity	$O(n)$

Section 2

Properties of Stacks – LIFO Principle

□ LIFO – LAST IN, FIRST OUT

The **LIFO principle** states that the element most recently inserted into the stack is the first one to be removed. This is the fundamental property that distinguishes a stack from all other linear data structures.

2.1 Core Properties

1

Single Access Point (Top): The stack has one end called the *Top*. All insertions and deletions occur exclusively at this end.

2

LIFO Ordering: The order of removal is the strict reverse of the order of insertion. If elements A, B, C are pushed in this order, they are popped as C, B, A.

3

Top Pointer: A special variable *top* (or *tos* – top of stack) always points to the most recently inserted element. Initially $top = -1$ (empty stack).

4

Bounded Capacity (Array): When implemented with an array, the stack has a maximum capacity (MAX). Inserting beyond capacity causes *Stack Overflow*.

5

Homogeneous Data: All elements in a stack are of the same data type.

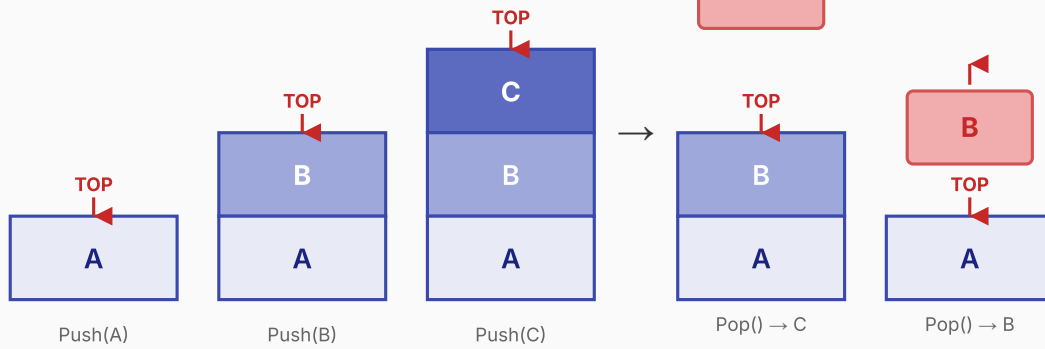
6

Sequential Storage: In array implementation, elements occupy contiguous memory locations.

2.2 LIFO Demonstration

FIG 2.1 – LIFO PRINCIPLE: PUSH A, B, C → POP C, B, A

PUSH SEQUENCE (A → B → C)



LIFO in action: Pushed A, B, C in order; Popped C, B, A in reverse order.

2.3 Comparison of LIFO vs FIFO

Property	LIFO (Stack)	FIFO (Queue)
Insertion at	Top	Rear
Deletion at	Top	Front
Order of removal	Reverse of insertion	Same as insertion
Access pattern	Most recent first	Oldest first
Implementation	One pointer (top)	Two pointers (front, rear)

★ KEY POINT

The LIFO principle is what makes stacks ideal for operations requiring reversal or backtracking. Any problem where the most recent state must be accessed first can be solved efficiently using a stack.

Section 3

Stack Operations – Push, Pop, Peek

3.1 PUSH Operation

DEFINITION

PUSH is the operation of inserting a new element at the *top* of the stack. Before pushing, the stack must be checked for overflow (whether it is already full).

Algorithm for PUSH

```
ALGORITHM – PUSH(STACK, ITEM)
```

```
// Pre-condition: top is the index of the topmost element  
BEGIN PUSH(stack, item)  
IF top == MAX - 1 THEN PRINT "Stack Overflow – Cannot insert" RETURN  
END IF  
top = top + 1 // increment top pointer  
stack[top] = item // insert element at new top  
PRINT "Item pushed: " + item  
END PUSH
```

FIG 3.1 – PUSH OPERATION: INSERTING ELEMENT 50



PUSH(50): top incremented from 2 to 3, element 50 placed at stack[3]

3.2 POP Operation

DEFINITION

POP is the operation of removing and returning the element from the *top* of the stack. Before popping, the stack must be checked for underflow (whether it is empty).

Algorithm for POP

ALGORITHM - POP(STACK)

```
BEGIN POP(stack) IF top == -1 THEN PRINT "Stack Underflow - Cannot delete" RETURN  
NULL END IF item = stack[top] // save the top element top = top - 1 // decrement  
top pointer RETURN item // return the removed element END POP
```

3.3 PEEK Operation

DEFINITION

PEEK (also called *Top*) returns the value of the top element **without removing it**. It is used to inspect the top without modifying the stack.

ALGORITHM - PEEK(STACK)

```
BEGIN PEEK(stack) IF top == -1 THEN PRINT "Stack is Empty" RETURN NULL END IF  
RETURN stack[top] // return without removing END PEEK
```

⚡ OPERATION SUMMARY

Push (Insert at top)	$O(1)$ time • $O(1)$ space
Pop (Remove from top)	$O(1)$ time • $O(1)$ space
Peek (View top element)	$O(1)$ time • $O(1)$ space
IsEmpty check	$top == -1 \rightarrow true$
IsFull check	$top == MAX-1 \rightarrow true$

Section 4

Stack Implementation Using Arrays

DEFINITION

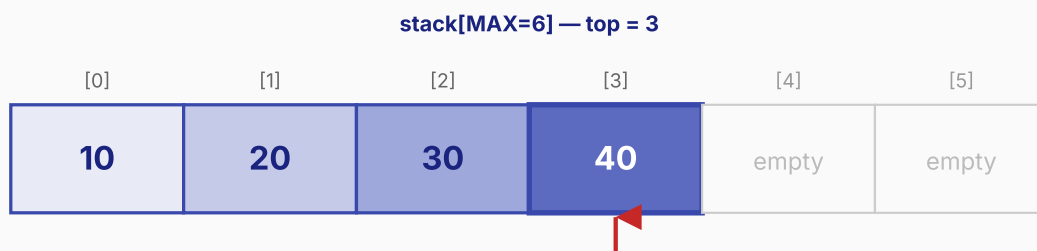
In array-based stack implementation, a fixed-size array is used to store elements. An integer variable `top` keeps track of the index of the most recently inserted element. Initially, `top = -1` indicating an empty stack.

4.1 Structure & Working Principle

The array-based stack consists of:

- **An array** `stack[MAX]` — stores the data elements
- **An integer** `top` — index of the topmost element (–1 when empty)
- **A constant** `MAX` — maximum number of elements the stack can hold

FIG 4.1 – STACK REPRESENTED AS AN ARRAY



top = 3 Available Space (4 & 5)
Elements stored in contiguous memory locations. `top` points to the last inserted element (index 3).

Array of size MAX=6 with 4 elements. `top=3` points to the most recent element (40).

4.2 Complete C Implementation

C PROGRAM – STACK USING ARRAY

```
#include <stdio.h> #define MAX 5 int stack[MAX]; int top = -1; // Initially stack
is empty /* — PUSH — */ void push(int item) { if (top == MAX - 1) { printf("Stack
Overflow!\n"); return; } top++; stack[top] = item; printf("Pushed: %d\n", item); }
/* — POP — */ int pop() { if (top == -1) { printf("Stack Underflow!\n"); return
-1; } int item = stack[top]; top--; return item; } /* — PEEK — */ int peek() { if
(top == -1) { printf("Stack Empty\n"); return -1; } return stack[top]; } /* — IS
EMPTY — */ int isEmpty() { return top == -1; } /* — IS FULL — */ int isFull() {
```

```
return top == MAX - 1; } int main() { push(10); push(20); push(30); printf("Top: %d\n", peek()); printf("Popped: %d\n", pop()); return 0; }
```

4.3 Advantages & Disadvantages

✓ Advantages

- Simple and easy to implement
- All operations are $O(1)$
- No extra memory for pointers
- Cache-friendly (contiguous memory)
- Fast direct access by index

✗ Disadvantages

- Fixed size — stack overflow possible
- Memory wasted if not fully used
- Cannot dynamically resize
- Size must be known in advance

Section 5

Stack Implementation Using Linked Lists

DEFINITION

In linked list-based stack implementation, each element of the stack is a node containing **data** and a **pointer to the next node**. The top pointer points to the first node (topmost element). This implementation allows dynamic sizing — the stack can grow or shrink as needed.

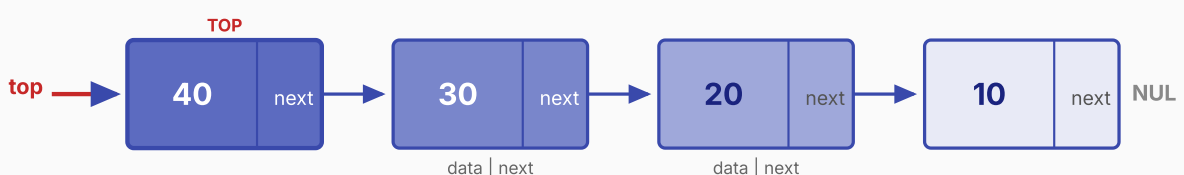
5.1 Node Structure

NODE STRUCTURE IN C

```
struct Node { int data; // data field struct Node* next; // pointer to next node };
struct Node* top = NULL; // top pointer - NULL means empty stack
```

5.2 Linked List Stack Diagram

FIG 5.1 – STACK USING LINKED LIST (TOP → 40 → 30 → 20 → 10 → NULL)



PUSH: new node before top

POP: remove top node

Linked list stack with 4 nodes. top always points to the most recently pushed node.

5.3 Complete C Implementation

C PROGRAM – STACK USING LINKED LIST

```
#include <stdio.h> #include <stdlib.h> struct Node { int data; struct Node* next;
}; struct Node* top = NULL; /* — PUSH — */ void push(int item) { struct Node*
newNode = (struct Node*)malloc(sizeof(struct Node)); if (newNode == NULL) {
printf("Heap Overflow\n"); return; } newNode->data = item; newNode->next = top; //
new node points to old top top = newNode; // top now points to new node
```

```
printf("Pushed: %d\n", item); } /* — POP — */ int pop() { if (top == NULL) {
printf("Underflow\n"); return -1; } int item = top->data; struct Node* temp = top;
top = top->next; // move top to next node free(temp); // free memory of old top
return item; } /* — PEEK — */ int peek() { if (top == NULL) { printf("Stack
Empty\n"); return -1; } return top->data; }
```

5.4 Advantages & Disadvantages

✔ Advantages

- Dynamic size — no fixed limit
- No stack overflow (limited by heap)
- Efficient memory use (grows as needed)
- Ideal for unpredictable input sizes

✖ Disadvantages

- Extra memory for pointer field in each node
- Slower due to dynamic memory allocation
- Not cache-friendly (non-contiguous)
- Risk of memory leak if not freed

5.5 Array vs Linked List Implementation

Parameter	Array Stack	Linked List Stack
Size	Fixed (static)	Dynamic (grows/shrinks)
Overflow	Possible when full	Only if heap is exhausted
Extra memory	None for pointers	One pointer per node
Access speed	Faster (cache locality)	Slightly slower
Implementation	Simple	Moderately complex
Time complexity	O(1) all ops	O(1) all ops

Section 6

Stack Overflow & Underflow

⚠ Stack Overflow

Occurs when a PUSH operation is attempted on a **full stack** (i.e., $top == MAX - 1$). No more elements can be inserted.

Condition:

$top == MAX - 1 \rightarrow$ Stack is FULL
PUSH fails $\rightarrow$ Overflow Error

⚠ Stack Underflow

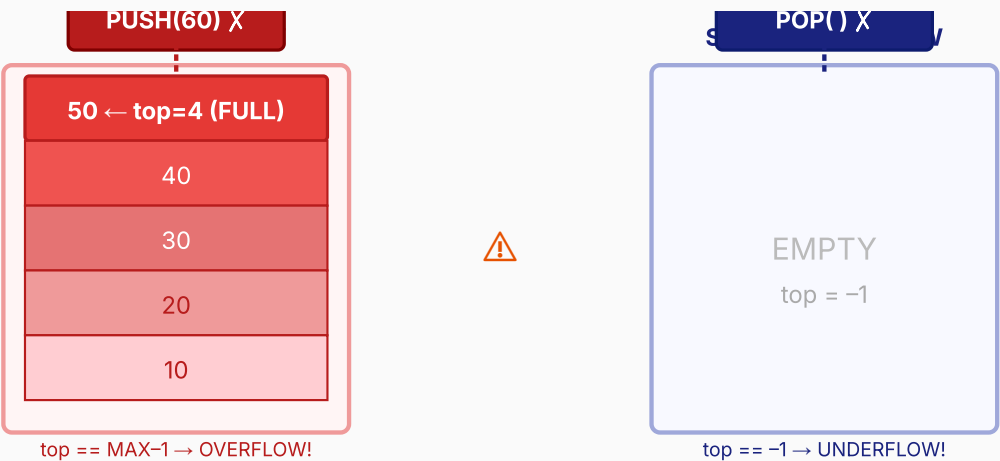
Occurs when a POP or PEEK operation is attempted on an **empty stack** (i.e., $top == -1$). No elements to remove.

Condition:

$top == -1 \rightarrow$ Stack is EMPTY
POP fails $\rightarrow$ Underflow Error

6.1 Overflow & Underflow Diagram

FIG 6.1 – STACK OVERFLOW AND UNDERFLOW CONDITIONS



6.2 Handling Overflow & Underflow

```
/* Check before PUSH */ if (top == MAX - 1){ printf("ERROR: Stack Overflow\n");  
exit(1); // or return error code } /* Check before POP */ if (top == -1) {  
printf("ERROR: Stack Underflow\n"); exit(1); // or return error code }
```

✦ KEY POINT

Always check for overflow before PUSH and underflow before POP. Failing to do so may cause **memory corruption** (array overflow) or **undefined behavior** in your program.

Section 7

Applications of Stacks

Stacks are among the most powerful and versatile data structures. Their LIFO behaviour makes them indispensable in numerous computing scenarios:

Expression Evaluation

Evaluating postfix (Reverse Polish Notation) expressions using a stack to hold operands and apply operators.

Expression Conversion

Converting infix expressions to postfix or prefix using operator precedence and a stack.

Backtracking

Storing states in maze solving, graph DFS traversal, game trees, and puzzle solving to retrace steps.

Reversing Data

Reversing strings, arrays, and linked lists by pushing all elements then popping them out.

Function Call Stack

The operating system uses a call stack to manage function calls, local variables, and return addresses for nested/recursive functions.

Parenthesis Matching

Validating balanced parentheses, brackets, and braces in source code using a stack.

Undo/Redo Operations

Text editors maintain two stacks — one for undo and one for redo — to track action history.

Browser History

Web browsers use a stack to store visited URLs. The Back button pops the last URL.

7.1 Application – Parenthesis Matching

ALGORITHM – BALANCED PARENTHESIS CHECK

```
BEGIN CheckBalance(expression)
FOR each character ch IN expression:
  IF ch is '(' OR '[' OR '{' THEN Push(ch)
  ELSE IF ch is ')' OR ']' OR '}' THEN
    IF stack is empty
```

```
THEN RETURN "UNBALANCED" top = Pop() IF top does not match ch THEN RETURN  
"UNBALANCED" END IF END FOR IF stack is empty THEN RETURN "BALANCED" ELSE RETURN  
"UNBALANCED" END CheckBalance
```

 APPLICATION SUMMARY

Application	Stack Role	Example
Expression Evaluation	Hold operands	3 4 + 2 * = 14
Infix → Postfix	Hold operators	A+B → AB+
Backtracking	Store visited states	Maze solving, DFS
Reversing	Push all, pop all	"HELLO" → "OLLEH"
Function calls	Store return address	Recursion
Undo/Redo	Action history	MS Word Ctrl+Z
Bracket matching	Match opening brackets	Compiler syntax check

Section 8

Expression Evaluation Using Stack

DEFINITION

Postfix (Reverse Polish Notation) expressions are evaluated using a stack. Each operand is pushed; when an operator is encountered, two operands are popped, the operation is performed, and the result is pushed back.

8.1 Postfix Evaluation Algorithm

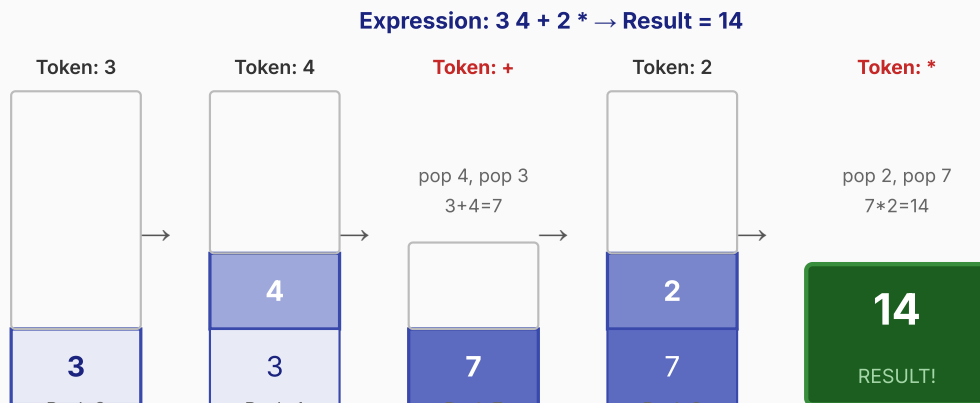
- 1 Scan the postfix expression from **left to right**.
- 2 If the current token is an **operand (number)**, push it onto the stack.
- 3 If the current token is an **operator (+, -, *, /)**, pop two operands from the stack (operand2 = pop, operand1 = pop).
- 4 Apply the operator: result = operand1 op operand2.
- 5 Push the result back onto the stack.
- 6 Repeat until all tokens are processed.
- 7 The final value on the stack is the **answer**.

8.2 Worked Example: Evaluate "5 3 2 * + 9 -"

Step	Token	Action	Stack (bottom → top)
1	5	Push 5	[5]
2	3	Push 3	[5, 3]
3	2	Push 2	[5, 3, 2]
4	*	Pop 2, 3 → 3*2=6 → Push 6	[5, 6]
5	+	Pop 6, 5 → 5+6=11 → Push 11	[11]
6	9	Push 9	[11, 9]
7	-	Pop 9, 11 → 11-9=2 → Push 2	[2]
8	END	Pop final result	Result = 2

8.3 Expression Evaluation Stack Diagram

FIG 8.1 – STACK STATE DURING POSTFIX EVALUATION OF "3 4 + 2 *"



Postfix expression "3 4 + 2 *" evaluates to 14. Stack state shown at each step.

8.4 Algorithm in C-style Pseudocode

POSTFIX EVALUATION – C PSEUDOCODE

```
int evaluatePostfix(char* expr) { int stack[MAX], top = -1; for (i = 0; expr[i] !=
'\0'; i++) { if (isdigit(expr[i])) stack[++top] = expr[i] - '0'; // push operand
else { val2 = stack[top--]; // second operand val1 = stack[top--]; // first operand
switch(expr[i]) { case '+': stack[++top] = val1 + val2; break; case '-':
stack[++top] = val1 - val2; break; case '*': stack[++top] = val1 * val2; break;
case '/': stack[++top] = val1 / val2; break; } } } return stack[top]; }
```

Section 9

Infix, Prefix & Postfix Conversions

9.1 Types of Expressions

Type	Operator Position	Example: A+B*C	Used by
Infix	Between operands	A + B * C	Humans / mathematicians
Prefix (Polish)	Before operands	+ A * B C	Some interpreters
Postfix (RPN)	After operands	A B C * +	Compilers, calculators

OPERATOR PRECEDENCE (HIGH → LOW)

^ (Exponentiation)	Highest – right to left
* and / (Multiplication, Division)	High – left to right
+ and – (Addition, Subtraction)	Low – left to right
() (Parentheses)	Overrides all precedence

9.2 Infix to Postfix Conversion Algorithm

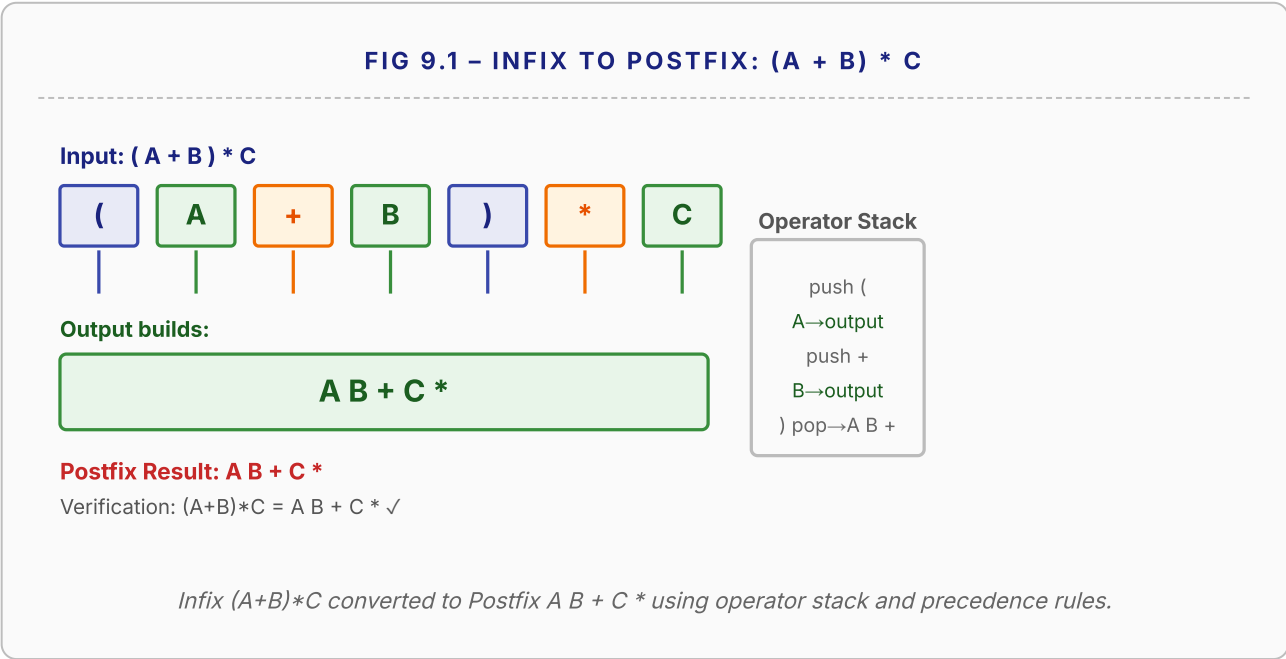
- 1 Scan the infix expression from **left to right**.
- 2 If the token is an **operand**, add it directly to the output.
- 3 If the token is '(', push it onto the stack.
- 4 If the token is ')', pop from stack and output until '(' is found. Discard both parentheses.
- 5 If the token is an **operator**, pop and output all operators on the stack with higher or equal precedence, then push the current operator.
- 6 At end of expression, pop and output all remaining operators from the stack.

9.3 Worked Example: A + B * C – D

Step	Token	Stack	Output (Postfix)	Action
1	A	[]	A	Operand → output
2	+	[+]	A	Push operator

3	B	[+]	A B	Operand → output
4	*	[+, *]	A B	* > + → push
5	C	[+, *]	A B C	Operand → output
6	–	[–]	A B C * +	Pop * and + (≥ precedence), push –
7	D	[–]	A B C * + D	Operand → output
8	END	[]	A B C * + D –	Pop remaining

9.4 Infix to Postfix Diagram



9.5 Conversion Examples Summary

Infix	Postfix	Prefix
A + B	A B +	+ A B
A + B * C	A B C * +	+ A * B C
(A + B) * C	A B + C *	* + A B C
A * B + C / D	A B * C D / +	+ * A B / C D
(A + B) * (C – D)	A B + C D – *	* + A B – C D

Section 10

Backtracking Using Stack

DEFINITION

Backtracking is a problem-solving technique that incrementally builds a solution and abandons (backtracks) a path as soon as it determines the path cannot lead to a valid solution. A stack is used to store states so the algorithm can retrace its steps.

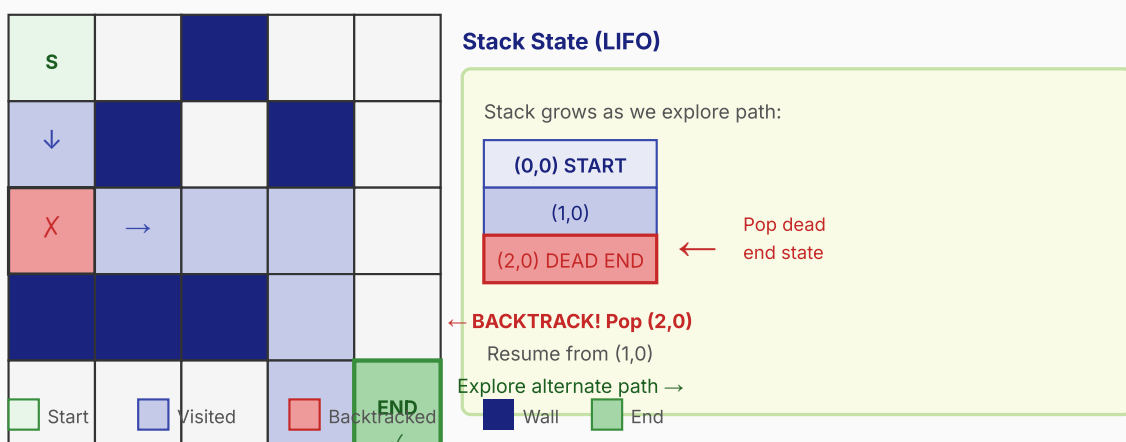
10.1 How Stacks Enable Backtracking

In backtracking, the stack acts as memory for previously explored states. When a dead end is reached, the algorithm pops the stack to retrieve the last valid state and tries an alternate path.

- **DFS (Depth First Search):** Uses a stack to explore nodes; backtracks when no unvisited neighbors remain.
- **Maze Solving:** Each cell visited is pushed; when stuck, pop to return to the previous cell.
- **N-Queens Problem:** Each queen placement is pushed; if invalid, pop and try the next column.

10.2 Backtracking Diagram – Maze Solving

FIG 10.1 – BACKTRACKING IN MAZE SOLVING USING STACK



Maze solving using DFS with backtracking. Dead ends are popped from the stack, allowing exploration of alternate paths.

10.3 DFS using Stack – Algorithm

DFS TRAVERSAL USING EXPLICIT STACK

```
BEGIN DFS(graph, startNode) Push(startNode) onto stack Mark startNode as visited
WHILE stack is not empty: current = Pop() PRINT current FOR EACH neighbor of
current: IF neighbor is not visited THEN Push(neighbor) Mark neighbor as visited
END IF END FOR END WHILE END DFS
```

Section 11

Reversing Data Using Stack

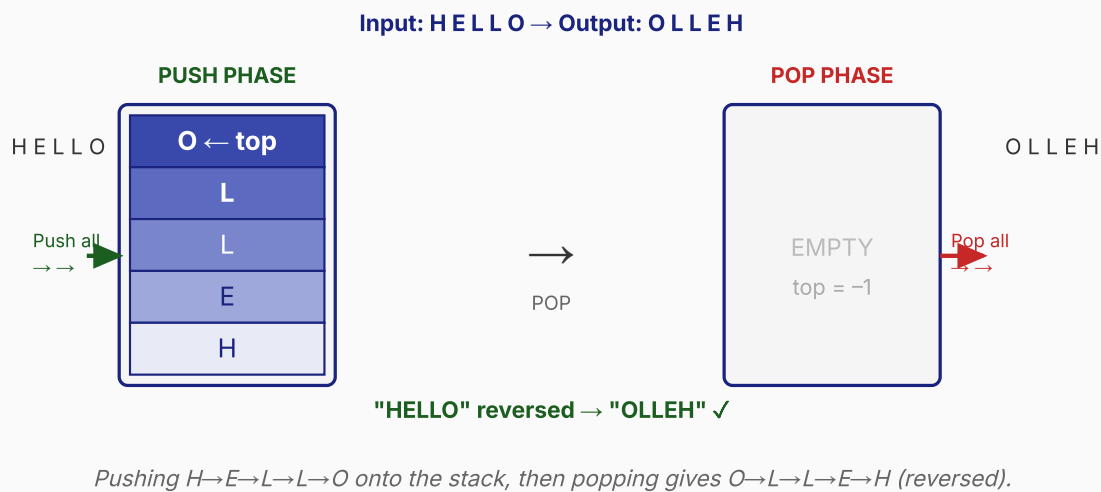
DEFINITION

The LIFO property of a stack makes it a natural tool for reversing data. Push all elements onto the stack, then pop them — the output is in reverse order of input. This works for strings, arrays, numbers, and linked lists.

11.1 String Reversal Using Stack

- 1 Read the string character by character from left to right.
- 2 Push each character onto the stack.
- 3 Once all characters are pushed, pop them one by one.
- 4 Each popped character is appended to the output — giving the reversed string.

FIG 11.1 – REVERSING "HELLO" USING STACK



11.2 Algorithm for Reversing a String

ALGORITHM – REVERSE STRING USING STACK

```
BEGIN ReverseString(str) n = Length(str) FOR i = 0 to n-1: Push(str[i]) // push
each character END FOR reversed = "" WHILE stack is not empty: reversed = reversed
+ Pop() // pop and append END WHILE RETURN reversed END ReverseString
```

11.3 Other Reversal Applications

Data to Reverse	Method	Example
String	Push chars, pop all	"abc" → "cba"
Array / List	Push elements, pop all	[1,2,3] → [3,2,1]
Number	Push digits, pop all	12345 → 54321
Linked List	Push nodes, pop & relink	1→2→3 becomes 3→2→1
Queue	Dequeue and push, then pop	Queue FIFO → reversed

⚡ COMPLEXITIES FOR REVERSAL

Time Complexity	$O(n)$ – n = number of elements
Space Complexity	$O(n)$ – stack stores all n elements
Push operations	$n \times O(1) = O(n)$
Pop operations	$n \times O(1) = O(n)$

Section 12

Call Stack Concept

DEFINITION

The **Call Stack** (also called the *Execution Stack* or *Program Stack*) is a special stack maintained by the operating system and runtime environment to keep track of active function calls, their local variables, parameters, and return addresses. Each function invocation creates a new **stack frame** (activation record) which is pushed onto the call stack.

12.1 Stack Frame Contents

Each Stack Frame Contains:

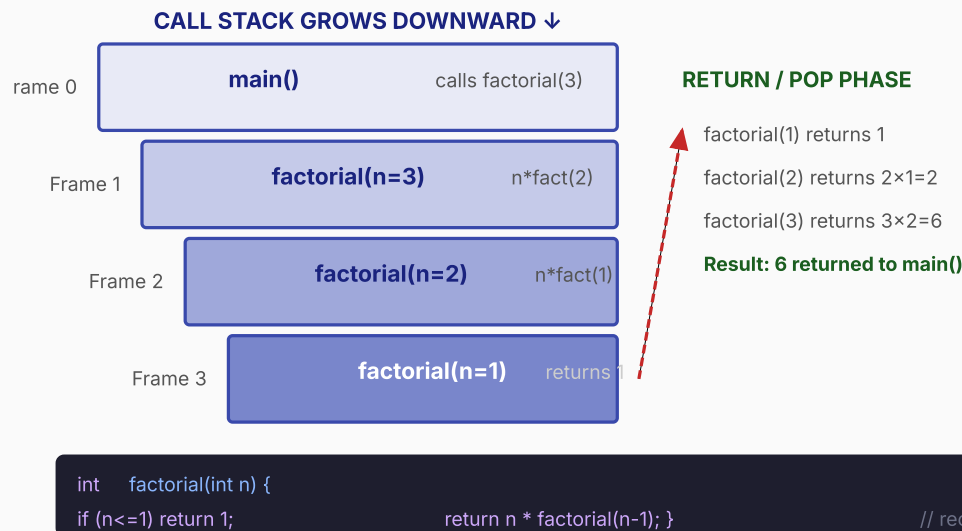
- **Return address** — where to go after function ends
- **Local variables** — declared inside the function
- **Parameters** — values passed to the function
- **Saved registers** — CPU register values
- **Base pointer** — reference to caller's frame

Call Stack Lifecycle:

1. Function is called → new frame PUSHED
2. Function executes with its local scope
3. If function calls another → another frame PUSHED
4. Function returns → frame POPPED
5. Control returns to previous frame

12.2 Call Stack Diagram – Recursive Factorial

FIG 12.1 – CALL STACK FOR FACTORIAL(3) = 6



factorial(3): 3 recursive frames pushed onto call stack. Returns unwind in LIFO order: 1→2→6.

12.3 Stack Overflow in Recursion

⚠ Infinite Recursion → Stack Overflow

If a recursive function lacks a proper base case, it calls itself indefinitely. Each call pushes a new frame, eventually exhausting the call stack memory and causing a **Stack Overflow Error** (e.g., `java.lang.StackOverflowError`, segmentation fault in C).

⚡ CALL STACK SUMMARY

Function call → Frame PUSH	O(1)
Function return → Frame POP	O(1)
Max stack depth (recursion)	Limited by OS/stack memory
Order of returns	Reverse of call order (LIFO)

Section 13

Questions & Answers

◆ 2-Mark Questions

Q1. What is a stack? State the LIFO principle.**2 Marks**

Stack is a linear data structure in which all insertions and deletions are performed at one end called the *top*. The **LIFO (Last In, First Out)** principle states that the most recently inserted element is the first to be deleted. Analogy: a pile of plates — the last plate placed on top is the first one taken. Initial state: $\text{top} = -1$ (empty stack).

Q2. Define Push and Pop operations.**2 Marks**

PUSH: The operation of inserting a new element at the top of the stack. Before pushing, a check for stack overflow ($\text{top} == \text{MAX}-1$) is performed. top is incremented and the element is stored at $\text{stack}[\text{top}]$.

POP: The operation of removing and returning the topmost element. Before popping, a check for stack underflow ($\text{top} == -1$) is performed. The element at $\text{stack}[\text{top}]$ is retrieved and top is decremented.

Q3. What is stack overflow and underflow?**2 Marks**

Stack Overflow: Occurs when a PUSH is attempted on a full stack. Condition: $\text{top} == \text{MAX} - 1$. No more space to insert elements. Program should report an error and abort the insertion.

Stack Underflow: Occurs when a POP or PEEK is attempted on an empty stack. Condition: $\text{top} == -1$. No elements to remove. Program should report an error and abort the deletion.

Q4. What is the time complexity of Push and Pop?**2 Marks**

Both **PUSH** and **POP** operations on a stack have a time complexity of **$O(1)$** — constant time. This is because both operations involve only a single step: incrementing or decrementing the top pointer and accessing one memory location. No traversal is needed. Space complexity for each individual operation is also $O(1)$.

Q5. What is postfix expression? Give an example.

2 Marks

Postfix expression (also called Reverse Polish Notation, RPN) is a notation in which operators are placed *after* their operands, eliminating the need for parentheses. The operator follows both operands.

Example: The infix expression $A + B * C$ is written in postfix as $A B C * +$ because $*$ has higher precedence and is applied to B and C first.

◆ 5-Mark Questions

Q6. Explain stack operations (Push, Pop, Peek) with diagram and algorithm.

5 Marks

Stack supports three primary operations:

1. PUSH Operation: Inserts a new element at the top. Algorithm: (1) Check if $\text{top} == \text{MAX}-1$ (overflow). (2) If no overflow, increment top: $\text{top} = \text{top} + 1$. (3) Store element: $\text{stack}[\text{top}] = \text{item}$. Time complexity: $O(1)$.

2. POP Operation: Removes the topmost element and returns it. Algorithm: (1) Check if $\text{top} == -1$ (underflow). (2) If not empty, save element: $\text{item} = \text{stack}[\text{top}]$. (3) Decrement top: $\text{top} = \text{top} - 1$. (4) Return item. Time complexity: $O(1)$.

3. PEEK/TOP Operation: Returns the topmost element WITHOUT removing it. Algorithm: (1) Check if $\text{top} == -1$ (empty). (2) Return $\text{stack}[\text{top}]$ without modifying top. Time complexity: $O(1)$.

Diagram Example: For a stack with $\text{MAX}=5$ containing $[10, 20, 30]$ ($\text{top}=2$): $\text{PUSH}(40) \rightarrow \text{top}$ becomes 3, $\text{stack}[3]=40$; $\text{POP}() \rightarrow$ returns 40, top becomes 2; $\text{PEEK}() \rightarrow$ returns 30, top remains 2.

All three operations are constant time $O(1)$, making stacks highly efficient for Last-In-First-Out access patterns.

Q7. Explain infix to postfix conversion using a stack with a worked example.

5 Marks

Infix to Postfix Conversion uses a stack to hold operators temporarily while operands go directly to the output.

Rules: (1) Operands $\rightarrow$ output directly. (2) '(' $\rightarrow$ push to stack. (3) ')' $\rightarrow$ pop until '(' found; output all, discard parentheses. (4) Operator $\rightarrow$ pop and output all stack operators with higher or equal precedence, then push current operator. (5) End of expression $\rightarrow$ pop and output all remaining stack operators.

Precedence: ^ (highest) $\rightarrow$ *, / $\rightarrow$ +, - (lowest).

Example: A + B * C

Token A: output = "A", stack = []

Token +: push, stack = [+], output = "A"

Token B: output = "A B", stack = [+]

Token *: * has higher precedence than +, push: stack = [+ , *], output = "A B"

Token C: output = "A B C", stack = [+ , *]

End: pop * $\rightarrow$ output = "A B C *", pop + $\rightarrow$ output = "A B C * +"

Postfix Result: A B C * + (equivalent to $A + B * C$, correctly respecting precedence)

Q8. Explain stack implementation using a linked list with algorithm and diagram.

5 Marks

Linked List Stack: Each element is a node with a data field and a next pointer. The top pointer always points to the most recently inserted (topmost) node. Initially top = NULL (empty stack).

Node Structure: struct Node { int data; struct Node* next; }; — data stores the value; next links to the previous element.

PUSH Algorithm: (1) Allocate a new node. (2) Set newNode $\rightarrow$ data = item. (3) Set newNode $\rightarrow$ next = top (new node points to old top). (4) Update top = newNode. This is $O(1)$.

POP Algorithm: (1) Check if top == NULL (underflow). (2) Save item = top $\rightarrow$ data. (3) Set temp = top. (4) Advance top = top $\rightarrow$ next. (5) Free temp. (6) Return item. This is $O(1)$.

Advantages over Array: Dynamic size (no overflow unless heap exhausted), efficient memory usage (no pre-allocation needed), and can grow to any size as per available memory.

Disadvantage: Extra memory per node for the pointer field; slightly slower due to dynamic memory allocation; not cache-friendly due to non-contiguous memory layout.

Stack using linked list: top $\rightarrow$ [50| $\rightarrow$] $\rightarrow$ [40| $\rightarrow$] $\rightarrow$ [30| $\rightarrow$] $\rightarrow$ [20| $\rightarrow$] $\rightarrow$ [10|NULL] demonstrates a stack with 5 elements where 50 is at the top.

Stacks have numerous practical applications in computer science:

- 1. Expression Evaluation:** Postfix expressions are evaluated using a stack. Operands are pushed; when an operator is encountered, two operands are popped, the result is computed and pushed. Example: "3 4 + 2 *" → result 14.
- 2. Expression Conversion:** Infix to Postfix/Prefix conversion uses a stack to temporarily hold operators while processing operand precedence. Compilers use this to parse mathematical expressions.
- 3. Backtracking:** Algorithms like DFS, maze solving, and constraint satisfaction use a stack to store explored states. When a dead end is reached, the stack is popped to backtrack to the previous valid state.
- 4. Reversing Data:** Strings, arrays, and linked lists can be reversed by pushing all elements and then popping them. LIFO guarantees reversal. Example: "HELLO" → "OLLEH".
- 5. Function Call Management (Call Stack):** The OS maintains a call stack where each function call creates a stack frame containing parameters, local variables, and return address. Recursive functions create multiple frames, all managed via the LIFO stack discipline.
- 6. Parenthesis Matching:** Compilers check for balanced parentheses by pushing opening brackets and popping when a closing bracket is found. Mismatch → syntax error.
- 7. Undo/Redo:** Text editors maintain an action stack; Ctrl+Z pops the last action and reverses it.

Section 14

Quick Revision Sheet

✦ ONE-PAGE REVISION GUIDE – READ BEFORE EXAM

This sheet summarizes all key concepts, operations, complexities, and rules for Unit III – Stacks.

📖 Key Definitions

- **Stack:** Linear LIFO data structure with single-end access (top)
- **Top:** Pointer to the most recently inserted element
- **LIFO:** Last In, First Out — most recent element removed first
- **Stack Frame:** Memory block for each function call
- **Overflow:** Push on full stack ($top == MAX-1$)
- **Underflow:** Pop from empty stack ($top == -1$)

⚡ Time Complexities

- Push $\rightarrow O(1)$
- Pop $\rightarrow O(1)$
- Peek/Top $\rightarrow O(1)$
- IsEmpty / IsFull $\rightarrow O(1)$
- Search $\rightarrow O(n)$
- Space (n elements) $\rightarrow O(n)$

🔄 LIFO Summary

- Push A, B, C $\rightarrow$ Pop gives C, B, A
- $top = -1 \rightarrow$ Empty stack
- $top = MAX-1 \rightarrow$ Full stack
- Only top is accessible
- No random access allowed
- Order: Reverse of insertion

🔧 Implementations

- **Array:** Fixed size, $O(1)$ ops, simple
- **Linked List:** Dynamic size, $O(1)$ ops, extra pointer
- Array: faster (cache friendly)
- Linked: no overflow (heap limit)
- Both: all ops are $O(1)$

🔺 Expression Conversion Quick Reference

OPERATOR PRECEDENCE (HIGH $\rightarrow$ LOW)

Exponentiation $^$	Highest Right Associative
Multiplication $*$, Division $/$	High Left Associative
Addition $+$, Subtraction $-$	Low Left Associative
Parentheses $()$	Override all precedence

Token Type	Infix→Postfix Action	Infix→Prefix Action
Operand	Add to output directly	Add to output directly
((open bracket)	Push to stack	Push to stack
) (close bracket)	Pop until (found; discard ()	Pop until (found; discard ()
Operator (curr)	Pop stack while $\text{prec}(\text{stack}) \geq \text{prec}(\text{curr})$; then push curr	Pop stack while $\text{prec}(\text{stack}) > \text{prec}(\text{curr})$; then push curr
End of expression	Pop and output all remaining	Reverse output

Example Conversions

- $A+B \rightarrow \text{Postfix: } AB+$
- $A+B*C \rightarrow AB\ C*+$
- $(A+B)*C \rightarrow AB+C*$
- $A*B+C/D \rightarrow AB*CD/+$

Applications Summary

- Expression evaluation (postfix)
- Infix $\leftrightarrow$ postfix/prefix conversion
- Backtracking & DFS
- Reversing strings/arrays
- Function calls (call stack)
- Parenthesis matching
- Undo/Redo operations
- Browser back button

Postfix Evaluation Rules

- Scan left to right
- Operand $\rightarrow$ Push
- Operator $\rightarrow$ Pop 2, compute, Push result
- Result = final stack top
- Note: operand2=first pop, operand1=second pop

Call Stack Facts

- Each function call $\rightarrow$ Push frame
- Function return $\rightarrow$ Pop frame
- Frame has: params + locals + return addr
- Recursion = multiple frames
- Infinite recursion $\rightarrow$ Stack Overflow

EXAM KEYWORDS TO REMEMBER

Stack is _____ data structure	LINEAR
Stack follows _____ principle	LIFO (Last In First Out)
Initial value of top	-1 (empty)
Overflow when top equals	MAX - 1
Best implementation for dynamic size	Linked List
Best implementation for fixed size	Array
Compiler uses stack for	Expression parsing, function calls

DFS traversal uses

Stack (explicit or call stack)

Textbook & Reference Books

Prescribed Textbooks

Mark Allen Weiss

Data Structures and Algorithm Analysis in C, Pearson Education.

Covers stack ADT (Chapter 3), expression evaluation, and stack-based applications in depth with rigorous algorithm analysis.

Ellis Horowitz & Sartaj Sahni

Fundamentals of Data Structures in C, Universities Press.

Comprehensive treatment of stacks (Chapter 3), including array and linked list implementations, and expression conversion algorithms.

Reference Books

Kurt Mehlhorn & Peter Sanders

Algorithms and Data Structures: The Basic Toolbox, Springer.

Provides a theoretical foundation for stack-based algorithms with formal complexity analysis.

Aho, Ullman & Hopcroft

Data Structures and Algorithms, Addison-Wesley.

Classic text covering stacks in the context of algorithm design, expression parsing, and compiler theory.

Brad Miller & David Ranum

Problem Solving with Algorithms and Data Structures Using Python, Franklin, Beedle & Associates.

Intuitive, example-driven exploration of stacks with hands-on coding exercises and real-world application contexts.

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest & Clifford Stein

Introduction to Algorithms (CLRS), 4th Edition, MIT Press.

The definitive algorithms textbook. Stack and queue structures covered in Chapter 10 with pseudocode and proofs.

Robert Sedgewick

Algorithms in C, Parts 1–4, Addison-Wesley.

Practical C implementations of stacks, queues, and linked lists with focus on real-world performance considerations.

Online Resources

Resource	Topic Coverage
GeeksForGeeks – Data Structures	Stack operations, implementations, applications
NPTEL – Data Structures (IIT)	Full lecture series on stacks and queues
Visualgo.net	Interactive stack visualisation and step-by-step simulation
CS50 Harvard (edX)	Memory, stack frames, and call stack concepts

✔ **UNIT III – STACKS: COVERAGE CHECKLIST**

- | | |
|--|--|
| ✔ Introduction to Stacks & LIFO principle | ✔ Infix to Postfix/Prefix conversions |
| ✔ Push, Pop, Peek operations with algorithms | ✔ Backtracking using Stack (DFS, Maze) |
| ✔ Array implementation with C code | ✔ Reversing data using Stack |
| ✔ Linked List implementation with C code | ✔ Call Stack & Function frames |
| ✔ Stack Overflow & Underflow conditions | ✔ 5 Q&A with 2-mark and 5-mark answers |
| ✔ Postfix Expression Evaluation | ✔ Quick Revision Sheet with all complexities |